

Análisis Reto: Bazar B2B

Proyecto B2B

1 de junio de 2026

Sumario

1. Aspectos Generales	2
1.1. Introducción	2
1.2. Problemática	2
1.3. Metodología o estrategia de desarrollo	2
2. Planificación y análisis	3
2.1. Análisis	3
2.2. Requisitos funcionales	3
2.3. Requisitos no funcionales	4
2.4. Historias de Usuario / Casos de Uso	4
2.5. Sprint	4
2.6. Diagramas: Clases y BD (diseño lógico) - Preguntas y respuesta al diagrama	5
2.7. Diagrama de componentes	10
2.8. Arquitectura lógica y física (propuesta)	11
2.9. Prototipo de pantallas no funcional	11

Capítulo 1

Aspectos Generales

1.1. Introducción

Bazar B2B es una plataforma transaccional mayorista (marketplace híbrido B2B y D2C) que busca conectar de manera eficiente toda la cadena de suministro de un producto. La plataforma permite que proveedores y mayoristas publiquen su mercancía para que tiendas o supermercados (minoristas) y el consumidor final puedan realizar pedidos de manera digital. El sistema actuará como un intermediario y catálogo 24/7, aplicando reglas de negocio específicas según el rol del usuario (precios escalonados, cantidades mínimas de pedido - MOQ).

1.2. Problemática

Actualmente, las compras al por mayor sufren de procesos manuales, fricciones en la verificación fiscal de los compradores (RUC/NIT) y falta de herramientas para establecer reglas estrictas de venta (como los márgenes de MOQ y tamaños de lote). Los proveedores necesitan una forma digital de validar a sus clientes empresariales y evitar que compras que no cumplan con el volumen mínimo se procesen, protegiendo así la rentabilidad de las ventas al por mayor sin recurrir a pasarelas bancarias complejas desde el inicio.

1.3. Metodología o estrategia de desarrollo

Se ha adoptado una metodología ágil orientada al desarrollo de un Producto Mínimo Viable (MVP) funcional para finales de agosto. Se hace un fuerte énfasis en evitar el crecimiento descontrolado del alcance (*Scope Creep*), desarrollando el sistema en Sprints lineales. La estrategia consiste en dividir la aplicación en 4 módulos épicos: Agregación de Oferta, Exhibición, Transacción y Gestión Operativa.

Capítulo 2

Planificación y análisis

2.1. Análisis

El análisis del sistema B2B se estructura en una cadena de valor de cuatro pasos: 1. Agregación de Oferta (Lado del proveedor). 2. Descubrimiento y Cotización (Lado de la tienda minorista). 3. Transacción y Gestión de Pedidos (El motor del marketplace). 4. Cumplimiento y Fidelización (Logística y postventa).

2.2. Requisitos funcionales

- **RF 1.1.1:** Creación y publicación de productos
- **RF 1.1.2:** Configuración del motor de reglas (MOQ y Lotes)
- **RF 1.1.3:** Asignación de precio base y minorista
- **RF 1.1.4:** Registrar el usuario
- **RF 1.1.5:** Registrar el minorista
- **RF 2.1.1:** Catálogo público navegable
- **RF 2.1.2:** Motor de precios dinámico (Role-Based Pricing)
- **RF 2.1.3:** Registro comercial con captura de RUC
- **RF 3.1.1:** Carrito de compras inteligente con bloqueo
- **RF 3.1.2:** Generación de Orden de Compra manual
- **RF 3.1.3:** Visualización de historial de pedidos
- **RF 4.1.1:** Panel de gestión de pedidos (Mayorista)
- **RF 4.1.2:** Bandeja de validación de usuarios (Superadmin)

2.3. Requisitos no funcionales

- **Rendimiento y SEO:** El Frontend debe implementar Renderizado del Lado del Servidor (SSR) obligatorio para el posicionamiento en motores de búsqueda (SEO).
- **Seguridad:** Se utilizarán JSON Web Tokens (JWT) para la comunicación segura entre cliente y servidor. Las reglas comerciales (como el MOQ) deben validarse de manera estricta en el Backend.
- **Mantenibilidad:** El sistema utilizará una arquitectura desacoplada para facilitar escalabilidad y futuras actualizaciones.

2.4. Historias de Usuario / Casos de Uso

1. **CU1 - Agregación de Oferta:** Como proveedor, quiero subir mi inventario y configurar el MOQ para evitar ventas no rentables.
2. **CU2 - Escaparate B2B:** Como minorista verificado, quiero navegar el catálogo logueado para descubrir precios de descuento exclusivos.
3. **CU3 - Transacción Inteligente:** Como sistema, quiero bloquear el carrito si el volumen del minorista no llega al MOQ.
4. **CU4 - Gestión Operativa:** Como administrador del *backoffice*, quiero validar el RUC de los nuevos usuarios para aprobar su rol B2B.

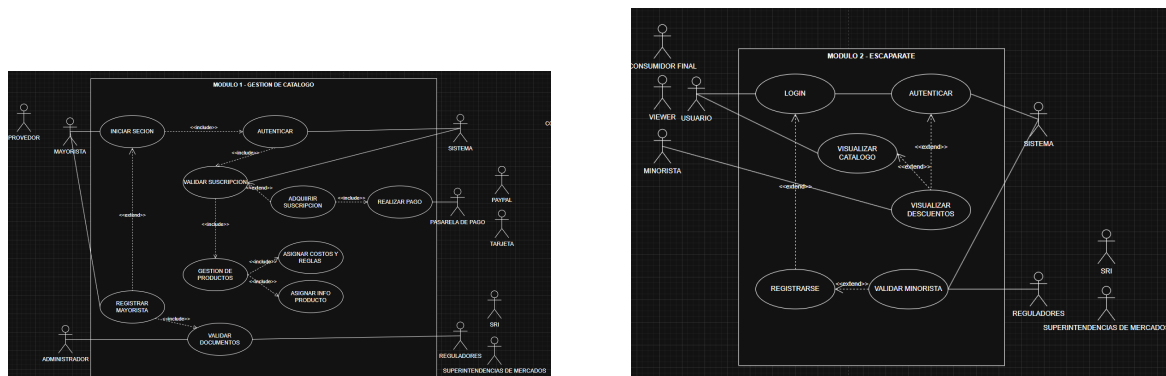


Figura 2.1: Casos de Uso 1 y 2 del Sistema Bazar B2B

2.5. Sprint

El primer Sprint se enfoca en el desarrollo del Catálogo Público con precios por Rol y el Motor de Carrito Inteligente (con MOQ validado), usando datos simulados (*Mock Data*) en una primera etapa para cumplir con los plazos de entrega a los auspiciantes.

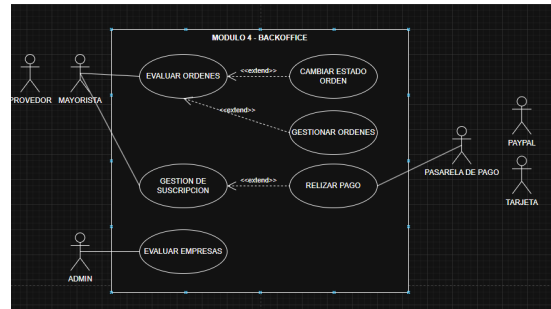
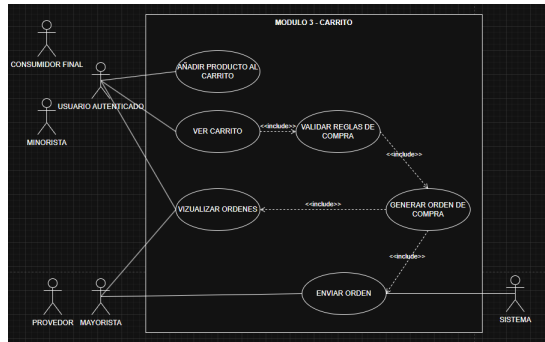


Figura 2.2: Casos de Uso 3 y 4 del Sistema Bazar B2B

2.6. Diagramas: Clases y BD (diseño lógico) - Preguntas y respuesta al diagrama

Diagrama de Clases

```
classDiagram
%% Relaciones
Usuario "1" -- "0..1" Suscripcion : tiene
PlanSuscripcion "1" -- "0..*" Suscripcion : genera
Usuario "1" -- "0..*" Producto : publica (Mayorista)
Usuario "1" -- "0..*" OrdenCompra : realiza (Cliente)
OrdenCompra "1" -- "1..*" DetalleOrden : contiene
Producto "1" -- "0..*" DetalleOrden : pertenece a
Producto "1" -- "0..*" ItemCarrito : en
Carrito "1" -- "0..*" ItemCarrito : agrupa
Usuario "1" -- "1" Carrito : posee

class Usuario {
+UUID id
+String nombre_empresa
+String email
+String password
+Enum rol (Admin, Mayorista, Minorista, Consumidor)
+String ruc_nit
+Enum estado_verificacion (Pendiente, Aprobado)
+iniciar_sesion()
+solicitar_verificacion()
}

class PlanSuscripcion {
+UUID id
+String nombre_plan
+Decimal precio_mensual
+Int limite_productos
+Boolean destacado
}
```

```

class Suscripcion {
    +UUID id
    +Date fecha_inicio
    +Date fecha_fin
    +Enum estado (Activa, Vencida)
    +procesar_pago()
    +verificar_vigencia()
}

class Producto {
    +UUID id
    +String sku
    +String titulo
    +Text descripcion
    +String imagen_url
    +Decimal precio_base
    +Decimal precio_minorista
    +Int moq_cantidad_minima
    +Int multiplo_lote
    +Int stock_disponible
    +calcular_precio_segun_rol(rol_usuario)
}

class Carrito {
    +UUID id
    +DateTime ultima_actualizacion
    +vaciar_carrito()
    +generar_orden_compra()
}

class ItemCarrito {
    +UUID id
    +Int cantidad_solicitada
    +Decimal subtotal
    +validar_regla_moq() Boolean
    +validar_multiplo_lote() Boolean
}

class OrdenCompra {
    +UUID id
    +String numero_orden
    +DateTime fecha_creacion
    +Enum estado_orden (Pendiente, Preparacion, Entregado)
    +Decimal total_calculado
    +String metodo_pago_acordado
    +cambiar_estado(nuevo_estado)
}

```

```

class DetalleOrden {
    +UUID id
    +Int cantidad
    +Decimal precio_unitario_congelado
    +Decimal subtotal
}

```

Pregunta 1: Restricciones de Suscripción en Alta Demanda B2B

Escenario: Un usuario se registra como *Mayorista*, pasa el proceso de validación y contrata un plan básico. Intenta ejecutar una carga masiva para publicar 400 productos en el catálogo, pero su plan tiene un límite estricto de 100 productos.

Pregunta: ¿Qué flujo exacto de clases, métodos y atributos interviene para autorizar su cuenta, procesar su membresía y posteriormente denegar la subida del producto número 101?

Resolución:

- **Quiénes intervienen:** Usuario, PlanSuscripcion, Suscripcion y Producto.
- **Por qué y cómo:**
 1. El Usuario (con atributo `rol = Mayorista` y `estado_verificacion = Pendiente`) invoca el método `solicitar_verificacion()`. Una vez aprobado (`estado_verificacion = Aprobado`), se le permite adquirir un plan.
 2. Se crea una instancia de `Suscripcion` a partir de un `PlanSuscripcion` seleccionado. El pago se consolida llamando a `Suscripcion.procesar_pago()`, lo que cambia su `estado` a `Activa`.
 3. Al momento de la carga masiva, por cada nuevo `Producto` que el Mayorista intenta instanciar (relación *Usuario 1 -> 0..* Producto*), el sistema debe verificar la cantidad de productos actuales asociados a ese ID de Usuario.
 4. El sistema cruza este conteo con el atributo `limite_productos` de la clase `PlanSuscripcion` vinculada a su `Suscripcion` activa. Al llegar al producto 101, la validación falla a nivel de negocio y se bloquea la creación de más instancias de `Producto`.

Pregunta 2: Colisión de Reglas de Venta B2B (MOQ y Lotes)

Escenario: Un *Minorista* está navegando por el catálogo e intenta agregar a su carrito 85 unidades de un suministro médico. El Mayorista configuró el producto con un `moq_cantidad_minima` de 50 unidades y un `multiplo_lote` de 20 unidades.

Pregunta: ¿Cómo reacciona el modelo de clases al intentar añadir estas 85 unidades y qué mecanismos garantizan que no se pueda evadir la regla antes de generar la orden de compra?

Resolución:

- **Quiénes intervienen:** Producto, Carrito y ItemCarrito.
- **Por qué y cómo:**

1. El **Carrito** del **Minorista** intenta crear una nueva instancia de **ItemCarrito** con el atributo `cantidad_solicitada = 85`.
2. Inmediatamente, el **ItemCarrito** debe invocar sus métodos de validación contra el **Producto** asociado:
 - Ejecuta `validar_regla_moq()`: Compara 85 contra el `moq_cantidad_minima` (50). Esta prueba es **exitosa** ($85 \geq 50$).
 - Ejecuta `validar_multiplo_lote()`: Verifica si 85 es divisible exactamente por el `multiplo_lote` (20). Esta prueba **falla** ($85 \% 20 \neq 0$).
3. Debido a que falla la regla de lote, la instancia del **ItemCarrito** es rechazada o entra en error, evitando que se sume al `subtotal`. En consecuencia, el método `generar_orden_compra()` de la clase **Carrito** no podrá procesar ítems inválidos.

Pregunta 3: Congelamiento de Precios e Inflación

Escenario: Un **Minorista** y un **Consumidor** añaden el mismo **Producto** a sus respectivos carritos el día lunes. El martes, el **Mayorista** actualiza el precio del producto subiéndolo un 15 %. El miércoles, ambos clientes deciden pagar y concretar la orden basándose en lo que vieron el lunes.

Pregunta: ¿De qué manera el diagrama resuelve la visualización de precios dinámicos por rol y cómo evita que el aumento de precio del martes afecte las órdenes procesadas el miércoles (o garantiza el precio histórico)?

Resolución:

- **Quiénes intervienen:** **Producto**, **Usuario**, **ItemCarrito** y **DetalleOrden**.

- **Por qué y cómo:**

1. **Cálculo dinámico:** El lunes, al visualizar el catálogo, se invoca el método `Producto.calcularPrecioSegunRol(rol_usuario)`. El sistema lee el atributo `rol` de cada **Usuario**. Al **Minorista** le retorna el `precio_minorista` y al **Consumidor** el `precio_base`.
2. **Transacción (El congelamiento):** El miércoles, cuando invocan `Carrito.generar_orden_compra()`, la aplicación debe recalcular el total al momento del *checkout*. Si la política del negocio es no respetar el precio del carrito antiguo, se vuelve a invocar `Producto.calcularPrecioSegunRol()`, tomando el nuevo precio con el 15 % de aumento.
3. **Persistencia:** Una vez efectuado el pago, los datos del **ItemCarrito** se transforman en una instancia de **DetalleOrden**. Aquí interviene un atributo vital: `precio_unitario_congelado`. Este atributo guarda el valor exacto pagado, independientemente de si el **Producto** cambia su precio futuro, garantizando la inmutabilidad de la **OrdenCompra** y protegiendo la auditoría financiera.

Pregunta 4: Ciclo de Vida de la Orden y Cancelaciones Tardías

Escenario: Un cliente paga por una orden (quedando en estado **Pendiente**). Horas después, el **Mayorista** inicia la preparación del envío. En ese mismo instante, el cliente intenta cancelar la compra desde su interfaz.

Pregunta: Basado en la clase `OrdenCompra`, ¿cómo se gestiona la máquina de estados de este documento y por qué la estructura de datos impide inconsistencias entre ambos actores?

Resolución:

- **Quiénes intervienen:** `OrdenCompra`, `Usuario` (Cliente) y `Usuario` (Mayorista/Admin).
- **Por qué y cómo:**
 1. La orden se instanció con un identificador único (UUID id) y el atributo `estado_orden` se inicializó en `Pendiente`.
 2. El Mayorista interviene ejecutando el método `cambiar_estado(nuevo_estado)`, actualizando el enum a `Preparacion`.
 3. Cuando el cliente intenta cancelar, el sistema debe consultar el estado actual de la `OrdenCompra`. Como el atributo ya mutó a `Preparacion`, la lógica de negocio detrás de `cambiar_estado()` evaluará que no se puede transicionar de `Preparacion` a un estado de cancelación (flujo no permitido directamente).
 4. El modelo lo resuelve de forma centralizada: al tener un solo estado (`estado_orden`) alojado en la cabecera del documento (`OrdenCompra`), asegura que no haya colisiones de estados paralelos entre el `Carrito` del cliente y el Backoffice del mayorista.

Pregunta 5: Carritos Abandonados y Retención de Stock

Escenario: Un cliente `Minorista` añade todo el stock disponible (1000 unidades) de un producto muy solicitado a su carrito, pero cierra el navegador sin comprar, dejando su sesión “en el limbo”. Otros compradores ven el producto como “Agotado”.

Pregunta: ¿Qué atributos presentes en el modelo de datos permiten a los procesos de Backoffice solucionar este cuello de botella y liberar el inventario retenido?

Resolución:

- **Quiénes intervienen:** `Carrito`, `ItemCarrito` y `Producto`.
- **Por qué y cómo:**
 1. El principal aliado para este escenario es el atributo `ultima_actualizacion` (tipo `DateTime`) ubicado en la clase `Carrito`.
 2. El sistema puede ejecutar una tarea programada (Cron Job) que escanee todas las instancias de `Carrito` buscando aquellas cuya `ultima_actualizacion` exceda un tiempo prudencial (ej. 24 horas sin actividad).
 3. Al detectar el carrito abandonado, el sistema interviene ejecutando el método `vaciar_carrito()`.
 4. Este método tiene un efecto en cascada: destruye las instancias de `ItemCarrito` vinculadas mediante la relación de agregación y toma la `cantidad_solicitada` que estaba retenida para sumarla nuevamente al atributo `stock_disponible` de la clase `Producto`, habilitando las ventas para otros clientes.

2.7. Diagrama de componentes

Arquitectura de Componentes

```
graph TD
    subgraph Frontend [Capa Frontend - Next.js]
        UI[Interfaz UI y Componentes]
        AuthFront[Gestor de Roles y Estado]
        Escaparate[Módulo Escaparate Público]
        Carrito[Módulo Carrito y MOQ]
        Backoffice[Dashboard Administrador]
    end

    subgraph Backend [Capa Backend - Django REST]
        API[API Gateway / Enrutador URLs]
        Security[Servicio Autenticación JWT]
        ProductSVC[Controlador de Catálogo y Reglas]
        OrderSVC[Controlador de Transacciones]
        ORM[Django ORM - Capa de Datos]
    end

    subgraph Database [Capa Persistencia]
        DB[(PostgreSQL)]
    end

    UI --> AuthFront
    UI --> Escaparate
    UI --> Carrito
    UI --> Backoffice

    API --> Security
    API --> ProductSVC
    API --> OrderSVC

    Security --> ORM
    ProductSVC --> ORM
    OrderSVC --> ORM

    Escaparate -.->|Consulta productos| API
    Carrito -.->|Envía órdenes| API
    Backoffice -.->|Gestión B2B| API
    AuthFront -.->|Valida Token| API

    ORM -->|Lectura / Escritura| DB
```

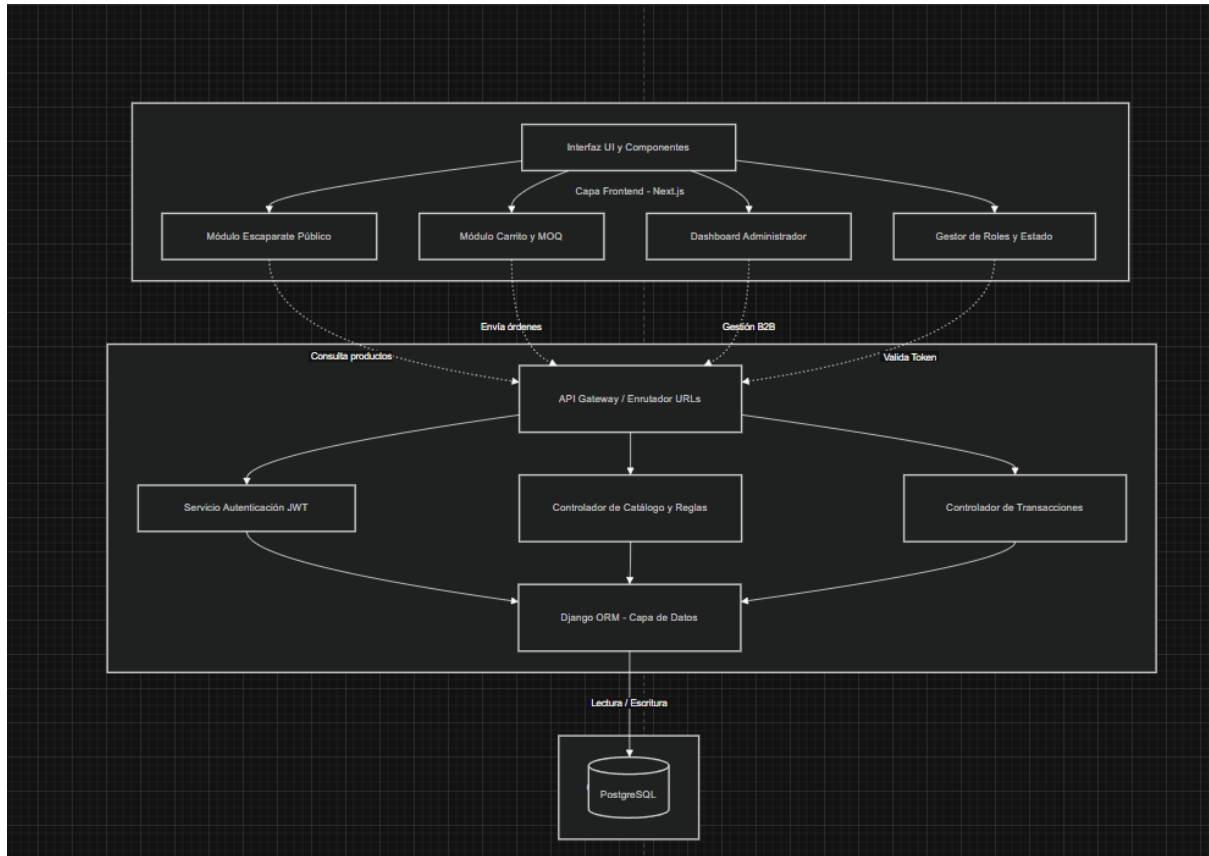


Figura 2.3: Diagrama de Componentes del Sistema Bazar B2B

2.8. Arquitectura lógica y física (propuesta)

Arquitectura lógica: Patrón MVC o Arquitectura Limpia, desacoplando la lógica de negocio del Backend en Python/Django, de la vista gestionada en Next.js/React.

Arquitectura física: Despliegue en la nube. Frontend alojado en servicios como Vercel y Backend/BD en Supabase o servidores cloud de rápida provisión (Render/AWS).

2.9. Prototipo de pantallas no funcional

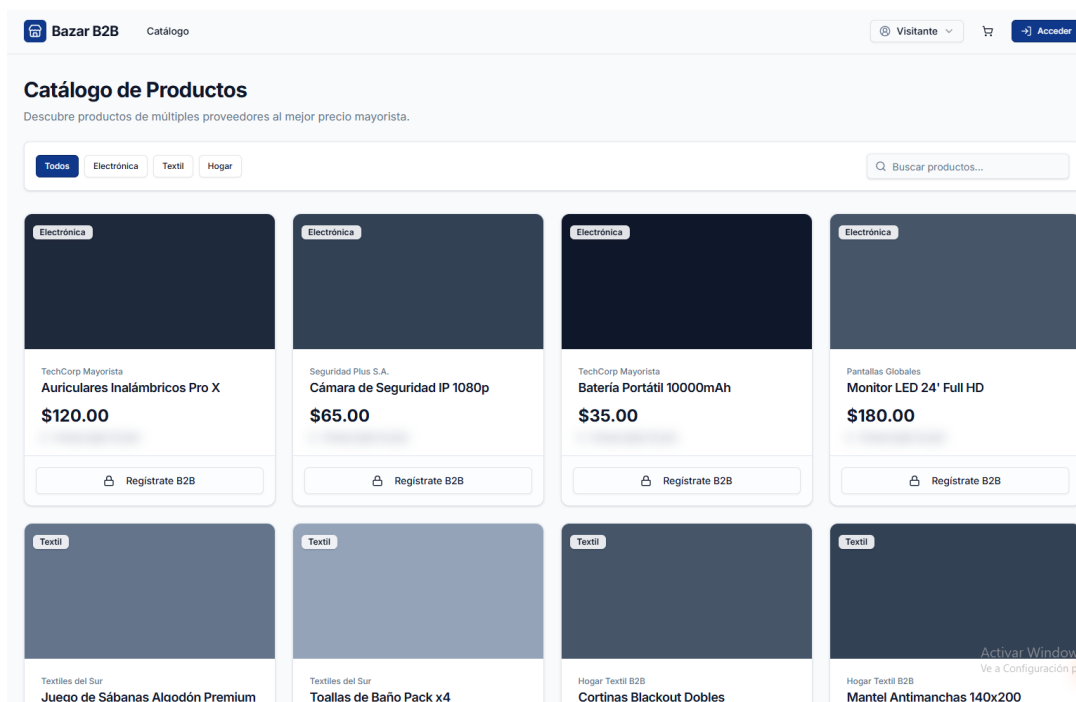


Figura 2.4: Prototipo 1: Vista de Catálogo Público

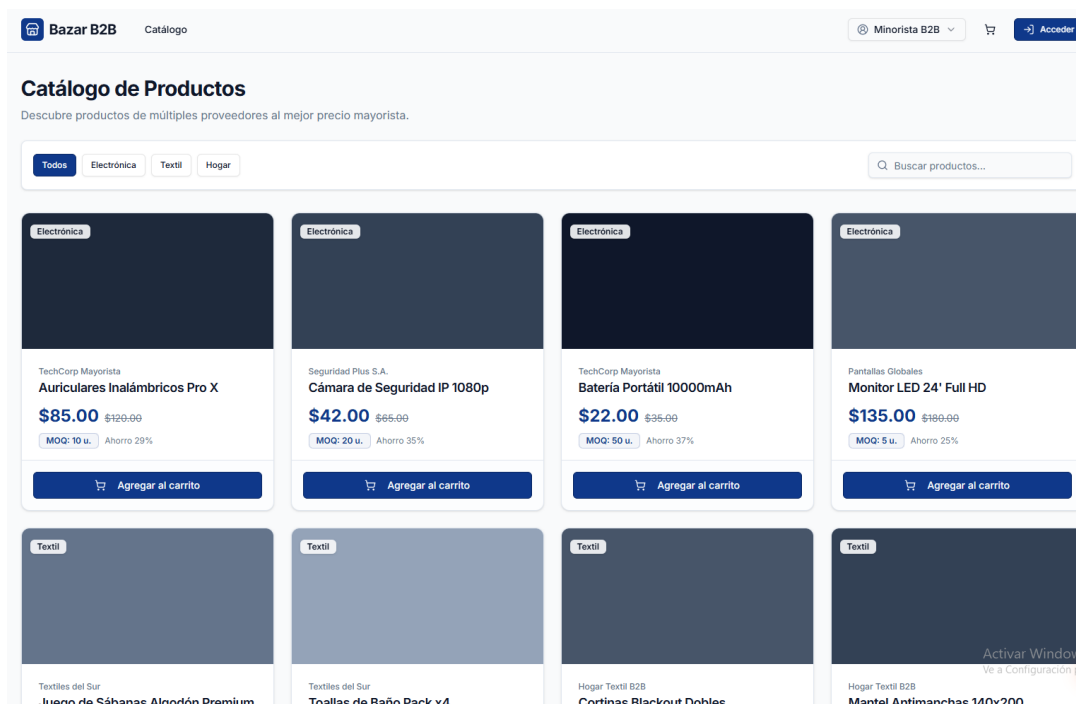


Figura 2.5: Prototipo 2: Carrito Inteligente MOQ y Checkout

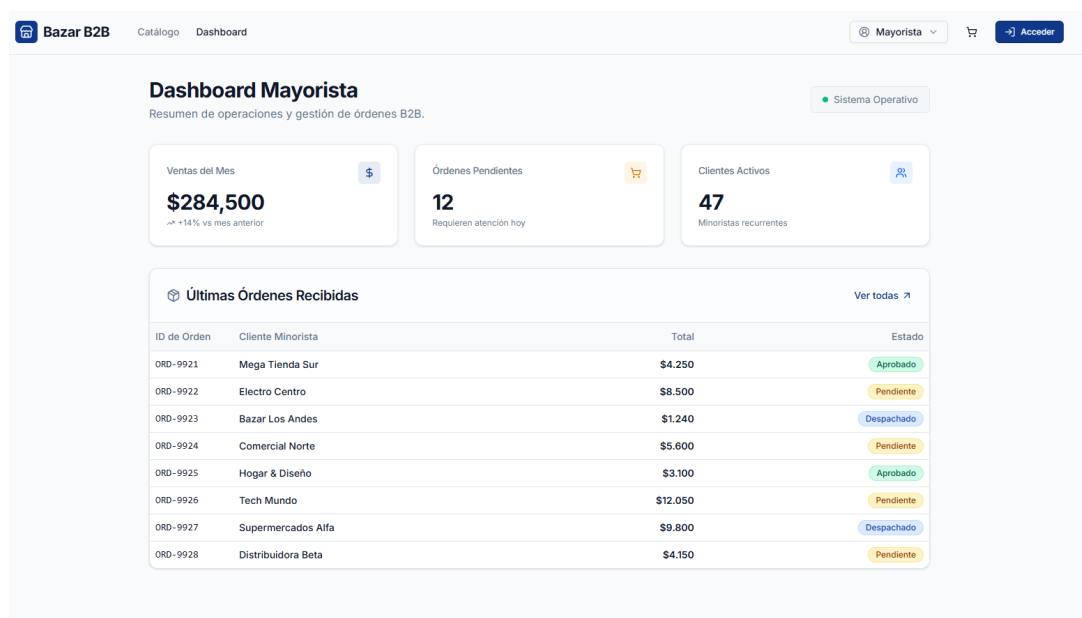


Figura 2.6: Prototipo 3: Dashboard Mayorista y Panel de Gestión